hashlock.

2024

Security Audit

ewe technology (Vault)

December 2024

info@hashlock.com.au

Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The ewe technology/N,B,A team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Benty is a mobile application that serves as a cryptocurrency wallet. It allows sending, receiving, swapping,+ and has a built-in cryptocurrency bridge functionality between multiple blockchains. Additionally, it is possible to stake your funds and watch transactions directly from the app. ewe technology/N,B,A is a decentralized finance (DeFi) revolution that is developing cutting-edge decentralized applications (dApps) and blockchain solutions. Our mission is to build secure, scalable, and innovative platforms that empower users to take control of their digital assets and participate in a decentralized financial ecosystem. With a focus on user experience and blockchain technology.

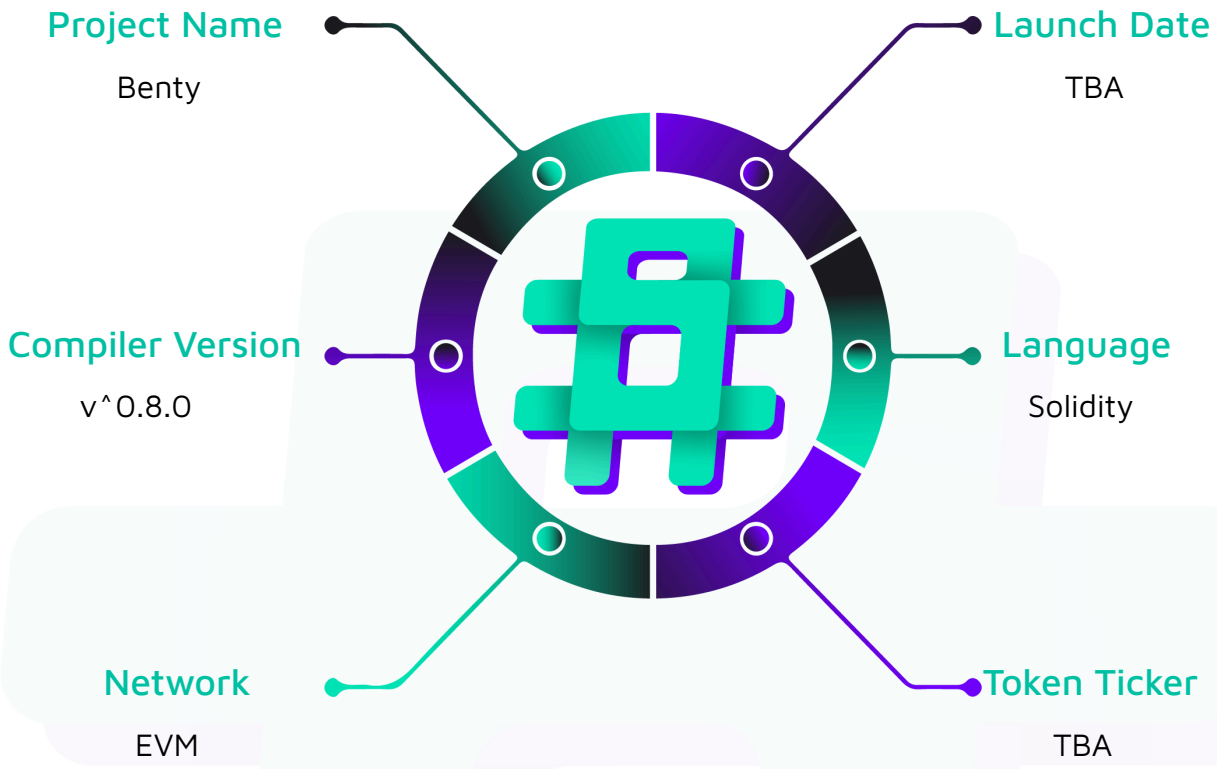
The smart contract does not have the ability for developers and third parties to freely transfer funds other than to add liquidity to Uniswap.

Project Name: Benty Wallet
Compiler Version: ^0.8.0
Website: https://ewetechnology.com/
Logo:

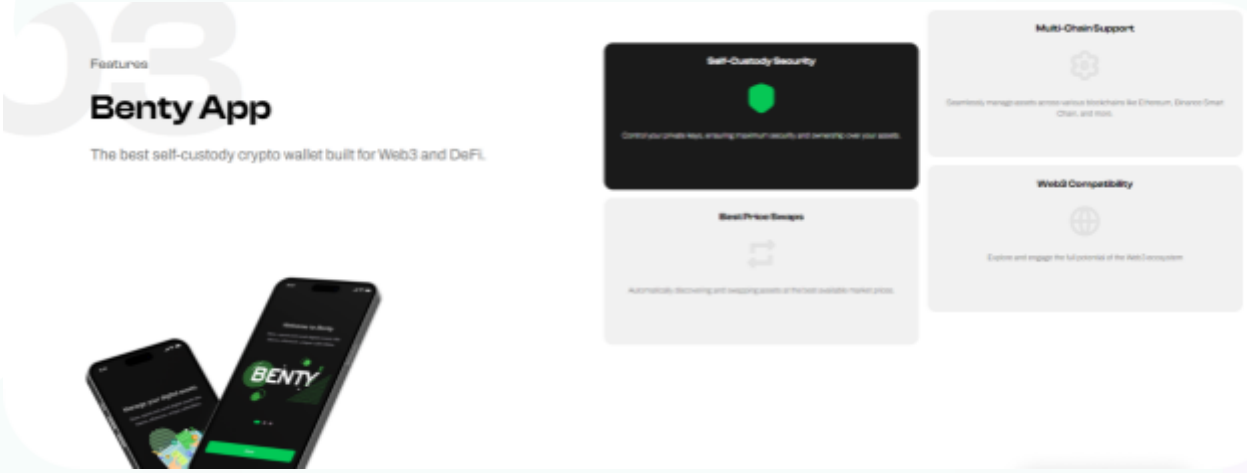
ewe technology

Next Blockchain Applications Limited

Visualised Context:



Project Visuals:



Audit scope

We at Hashlock audited the solidity code within the Benty project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	ewe technology Smart Contracts
Platform	EVM / Solidity
Audit Date	December, 2024
Contract 1	DCAManagement.sol
Contract 1 MD5 Hash	06da8732175b4545e4efc57e3c14d754
Contract 2	DCAVault.sol
Contract 2 MD5 Hash	2b62824f34112d9f0f6d2f41c5aeb160
Contract 3	Farm.sol
Contract 3 MD5 Hash	94f9a1b9b1c621f47b97b0e1e4ccd231
Contract 4	ReferralManagement.sol
Contract 4 MD5 Hash	53c275f52283a26fc3ccd03b075fee99
Contract 5	ReferralVault.sol
Contract 5 MD5 Hash	ce25dae378b77cd891572a5b159363f2
Contract 6	StrategyUserManager.sol
Contract 6 MD5 Hash	5ac5a68e27a21d14e8010e9ebf5c1b33
Contract 7	StrategyV11.sol
Contract 7 MD5 Hash	562080a16c807dfdf6d854b9b28a7635

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts. We initially identified some significant vulnerabilities that have since been addressed.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The general security overview is presented in the Standardised Checks section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 Medium severity vulnerability

2 Low severity vulnerabilities

2 Gas Optimisations

2 QAs

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour

DCAManagement.sol
- Allows users to:
  - Create DCA plan
  - Update DCA plan
  - Claim reward
  - Cancel a specific DCA
- Allows EXECUTOR_ROLE to:
  - Execute the DCA plan for a specific strategy by user
  - Set fee numerator
  - Withdraw the token
	Contract achieves this functionality.

DCAVault.sol - Allows management contract to: - Transfer tokens and native coins to the recipient	**Contract achieves this functionality.**

Farm.sol
- Allows users to:
  - Deposit liquidity with one of the pair token
  - Deposit USDC for liquidity
  - Withdraw liquidity
  - Claim reward
	Contract achieves this functionality.

ReferralManagement.sol
- Allows users to:
  - Add a referral relationship between a referrer and a referee
  - Claim reward
- Allows EXECUTOR_ROLE to:
	Contract achieves this functionality.

- Update the referrer of a referee - Set the referral percentage for a referrer - Set a default percentage numerator	
ReferralVault.sol - Allows management contract to: - Transfer tokens to the recipient	**Contract achieves this functionality.**
StrategyV11.sol - Allows INVESTOR_ROLE to: - Deposit liquidity - Withdraw liquidity - Claim reward - Allows CONTROLLER_ROLE to: - Collect rewards - Earn preparation - Rescale - Deposit dust tokens	**Contract achieves this functionality.**

Code Quality

This audit scope involves the smart contracts of the Benty project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Benty project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Severity Definitions

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are purely informational and don't impact functionality. These notes help clients improve the clarity, maintainability, or overall structure of the code, ensuring a cleaner and more efficient project. They should be addressed for optimization but are not critical to the system's performance or security.

Audit Findings

Medium

[M-01] StrategyUserManager#addUserToListOnFirstDeposit - The first user's index is updated to 0 which represents removed users

Description

The `addUserToListOnFirstDeposit` function updates the first user's index to 0 and the zero index represents the removed users.

Vulnerability Details

The `addUserToListOnFirstDeposit` function updates the `userIndex[userAddress]` to `userList.length`. But if the caller is the first user, `userList.length` is still 0 and `userIndex[userAddress]` is updated to 0.

```
function addUserToListOnFirstDeposit(address userAddress) internal {
    ...
    userIndex[userAddress] = userList.length;
    ...
}
```

So the first user's index could be considered as a removed user.

Recommendation

Consider starting the users' index from 1 instead of 0 if the zero is intended to represent removed users.

Status

Resolved

Low

[L-01] DCAManagement#createDCAPlan - Every new strategy creation calls add() that adds the user to allUsers even if the user is not a new user

Description

The users are allowed to have multiple strategies and the `createDCAPlan` function allows users to create new plans to their strategies.

The users can add plans to new strategies as well.
However, the function tries to add the user address to `allUsers` whenever the user creates new strategies even if the user is not a new user.

```
function createDCAPlan(
    address _strategy,
    address _inputToken,
    uint256 _eachTimeInvestAmount,
    uint256 _eachTimePeriod,
    DCAPeriodData calldata _periodData
) public {
    ...
    bool isNewStrategy = userAllStrategies[user].add(_strategy);
    if (isNewStrategy) {
      allUsers.add(user);
    }
    ...
}
```

Recommendation

Consider adding the user to `allUsers` only if `userAllStrategies[user].length` is 1.

Status

Resolved

[L-02] DCAManagement#setFeeNumerator, setMaxPlanAmount, StrategySetter#setTransactionDeadlineDuration, setBuyBackToken, setBuyBackNumerator, setEarnLoopSegmentSize, ReferralManagement#setDefaultPercentageNumerator - Missing events

Description

The above setters functions are missing events when key states are updated.

Recommendation

Add relevant events in the functions.

Status

Resolved

Gas

[G-01] StrategyV11#earnPreparation - Cache array length before looping

Description

Before iterating through an array with a for-loop, the length of the array can be cached in memory so that its value does not need to be accessed on each iteration of the loop.

Recommendation

Cache the array length by assigning it to a variable in memory before looping through the array.

Status

Resolved

[G-02] ReferralVault - managementContract could be made immutable

Description

The `managementContract` variable is only updated in the constructor.

The same for the `Constants` variable for all the contracts.

Recommendation

Make the `managementContract` variable immutable.

Status

Resolved

QA

[Q-01] Contracts - Floating pragma

Description

The contracts have `pragma solidity ^0.8.0` and it might allow the contracts to be deployed with a different version than the one used for testing.

Different pragma versions being used in test and mainnet may pose unidentified security issues.

Recommendation

Specify a specific version of Solidity in the pragma statement.

Status

Resolved

[Q-02] DCAManagement - Unnecessary assignment to a default value

Description

The contract assigns the `feeNumerator` variable to 0 which is its default value during the deployment.

Recommendation

Remove the assignment to 0.

Status

Resolved

Conclusion

After Hashlock's analysis, the Benty project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au